

# Modern AI systems: agents and system optimizations

## Agent Design II: Multi-Agent Systems

Juncheng Yang



**Harvard** John A. Paulson  
**School of Engineering**  
and Applied Sciences



HARVARD UNIVERSITY

**MaDSys**

Measurement and Design of Systems Lab

# Logistics

- Claude Code Premium seat
- Assignment 1
  - Canvas
  - repo url
  - mini product
- Assignment 2 is waiting for you, due in ~two weeks
- Office hour next Thursday 3:30-5pm SEC 4.308
- Pre-proposal

# Plan for today

- Multi-agent design
  - Architecture
  - Coordination: task decomposition
  - Communication
- Agent memory



# From one agent to many

- Architecture
- Coordination
- Communication

Multi-agent is not new, it has decades of research history which we will not get into



# When and why split work across agents?



## **specialization**

different models, prompts and tools



## **isolation**

give different agents different privileges



## **parallelism**

independent subtasks execute concurrently



## **context management**

don't give every agent the entire trajectory

# Three questions / graphs behind every architecture

## **Control graph: Who coordinates?**

One supervisor, a hierarchy, or nobody at all, does it change during runtime (handoff)

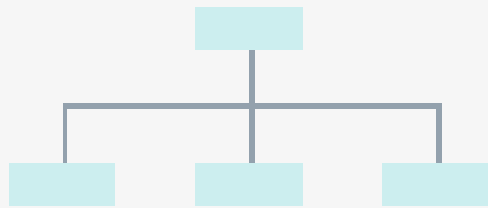
## **Task graph: How is work decomposed?**

Up front by a planner, by bidding, or emergently from local rules

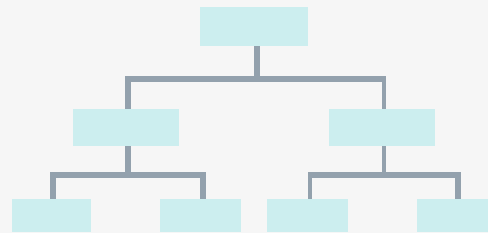
## **Communication graph: How do they communicate?**

Direct messages, broadcast, or a shared workspace

# Multi-agent architecture



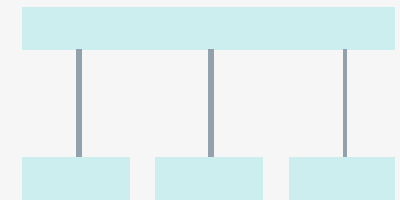
1 Supervisor-worker



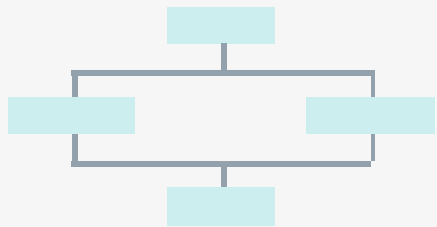
2 Hierarchical



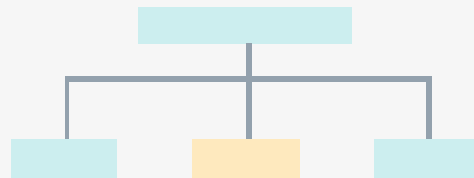
3 Pipeline / DAG



4 Blackboard



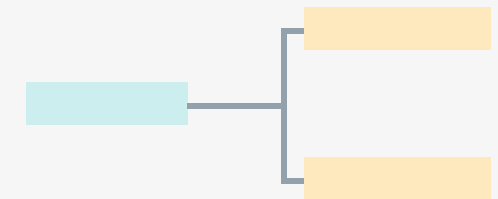
5 Committee



6 Market



7 Swarm



8 Dynamic topology

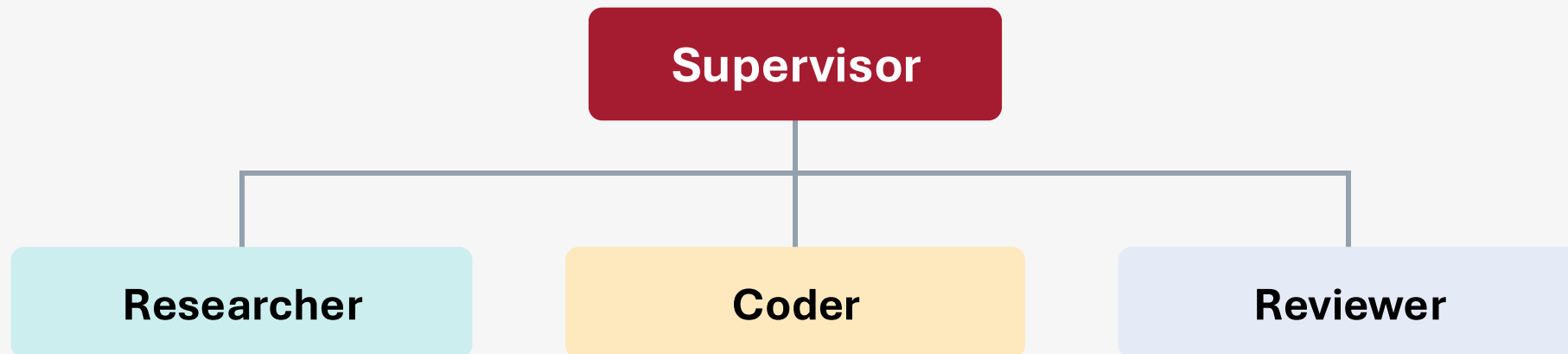


# 1. Supervisor–worker

- One agent plans, delegates and integrates; workers see only their subtask

Best when subtasks are independent and can run in parallel

**Weakness:** the supervisor bottlenecks: it re-reads every result and its context grows fastest.



**Task:** supervisor splits up front · **Control:** one supervisor · **Comms:** star, via supervisor

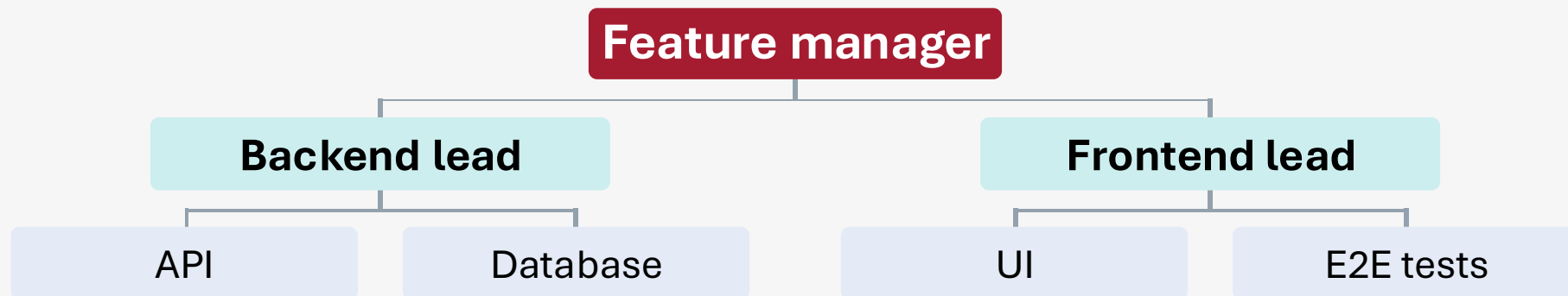
## 2. Hierarchical agents

- Supervisor–worker applied recursively: supervisors manage sub-supervisors

Fits tasks that already decompose hierarchically — storage, query engine, testing

**Challenge:** where does decomposition happen, and how much context crosses each boundary?

**Example:** feature build



**Task:** split again at each level · **Control:** nested supervisors · **Comms:** tree, parent–child only

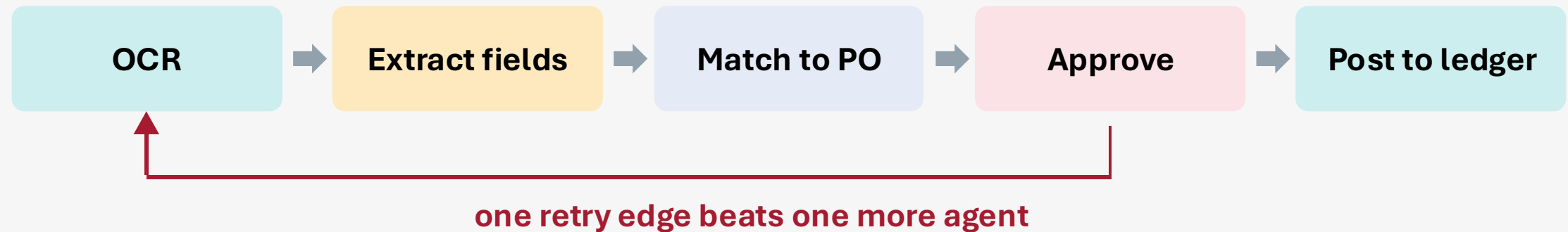
### 3. Pipeline / DAG

- Fixed stages with a typed contract at each boundary

Cheapest topology to debug — errors localize to a stage

**Weakness:** no path backwards; stage four cannot ask stage one for a better source.

**Example:** invoice processing



**Task:** stages fixed at design time · **Control:** the order is the control · **Comms:** stage to stage

## 4. Blackboard and shared memory

- Agents coordinate through shared state instead of direct messages
- The workspace may be a database, vector store, filesystem, Git repo or task graph

Agents can join or leave without rewiring; the shared state is durable and inspectable

**Challenges:** consistency, concurrency control, provenance, versioning, garbage collection.



**Task:** emerges from shared state · **Control:** none central · **Comms:** indirect, via workspace

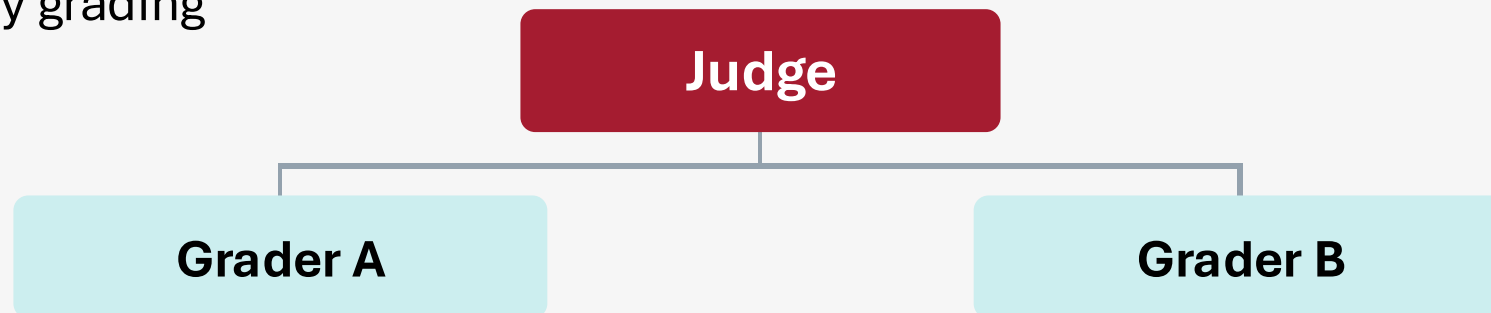
## 5. Committee / debate

- Agents propose and discuss, then a judge or vote resolves the disagreement

Improves accuracy on judgment-heavy tasks

**Failure mode** — agreement without independence: same model and context, same blind spot.

**Example:** essay grading



**Task:** one task, repeated in parallel · **Control:** the judge decides · **Comms:** peers, then to judge

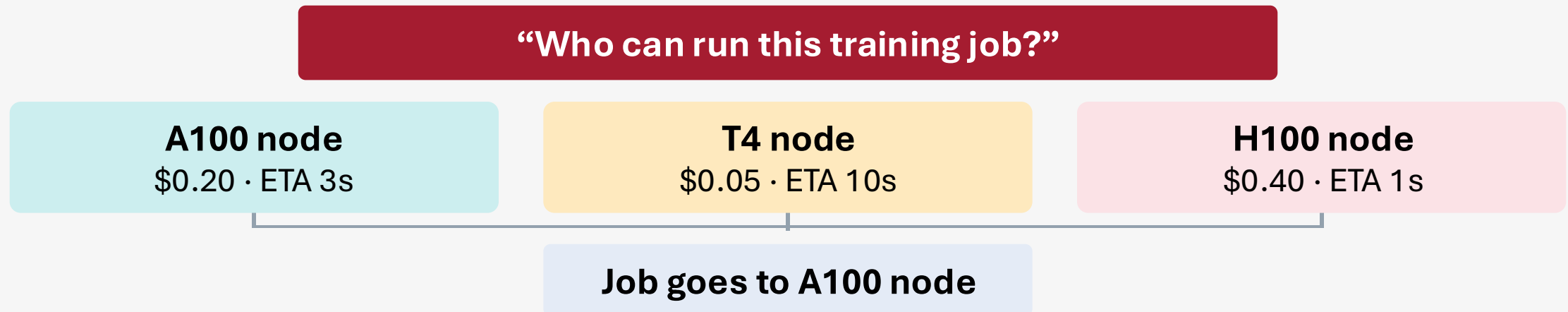
## 6. Market and auction

- Agents bid for announced tasks based on capability, cost and latency
- Heterogeneous agents differ in models, tools, machines, cost and expertise

Work lands on the cheapest capable agent with no central planner

**Weakness:** bids are self-reported, so an agent can win by overstating what it can do

**Example:** GPU job router — cheapest bid under 5s wins



**Task:** announced, claimed by bid · **Control:** auctioneer clears · **Comms:** broadcast, bids back

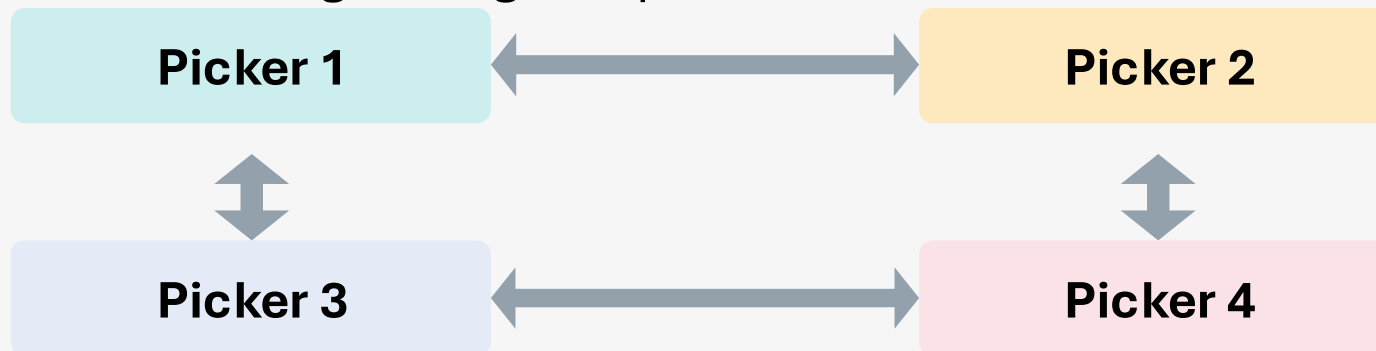
# 7. Swarm

- No permanent coordinator, agents negotiate and exchange findings directly
- Needs protocols for discovery, addressing, synchronization, termination, trust

No coordinator to bottleneck or fail; work continues as peers drop in and out

**Weakness:** communication complexity  $O(n^2)$ ; conflict resolution are hard. Rarely used in products.

**Example:** warehouse robots negotiating who picks which order



**Task:** negotiated locally · **Control:** no permanent coordinator · **Comms:** all-to-all,  $O(n^2)$

## 8. Dynamic topology

- Agents are added, removed or rewired while the task is running

Capacity scales with difficulty; simple tickets stay cheap

**Weakness:** every run takes a different shape, so failures are hard to reproduce

**Example:** support bot adds billing and log agents only when a ticket escalates.



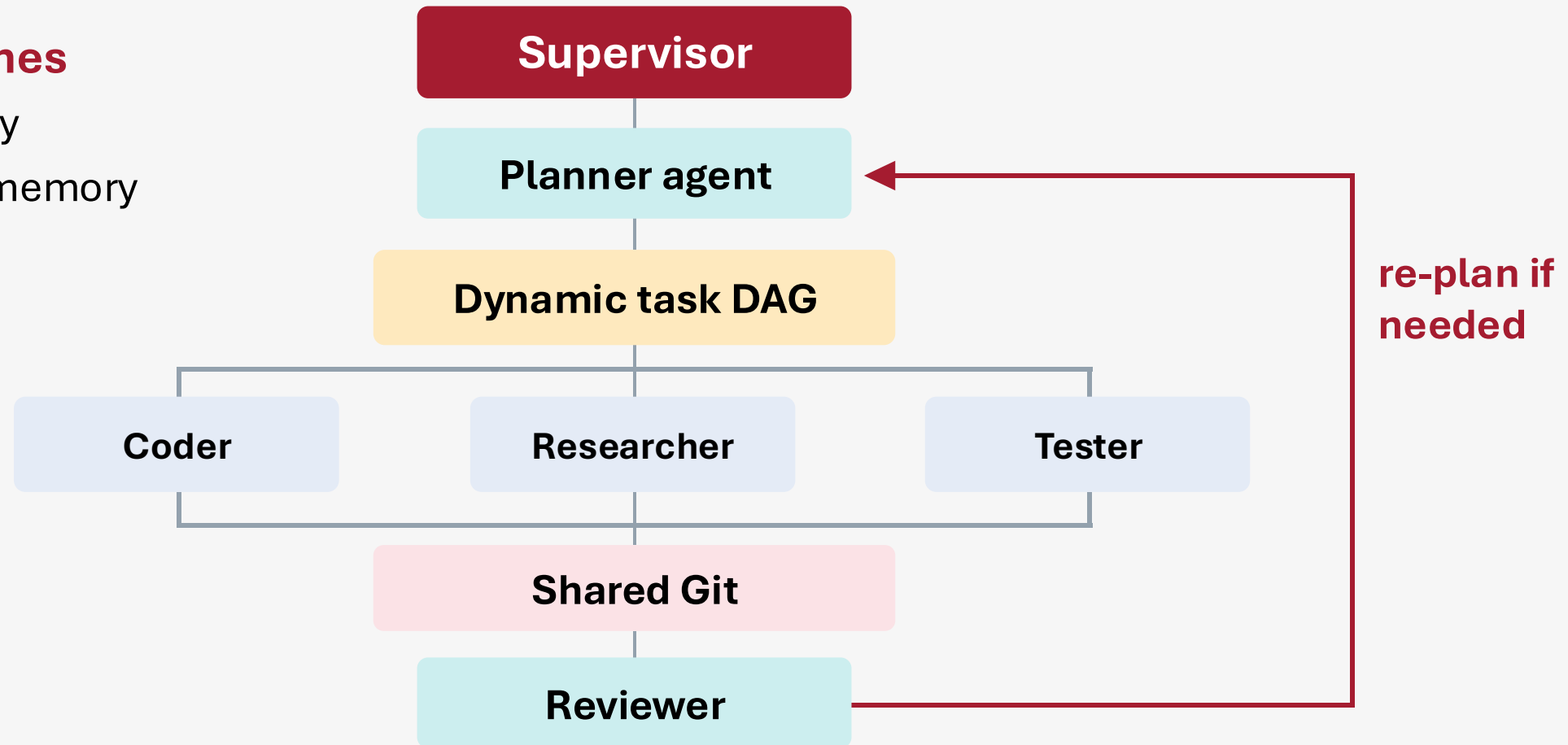
**Task:** re-split as work gets hard · **Control:** shifts as agents join · **Comms:** dynamic at runtime



# Real systems are hybrids

## Combines

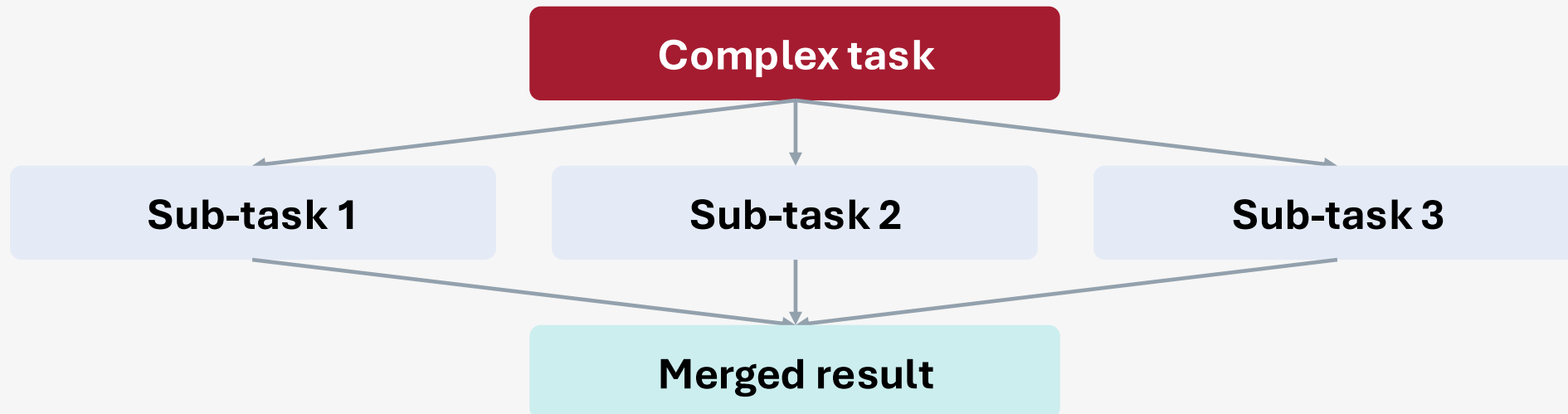
hierarchy  
shared memory  
pipeline



# Eight architectures

| Category      | Architecture              | Core idea                  | Best for             | Weakness           |
|---------------|---------------------------|----------------------------|----------------------|--------------------|
| Centralized   | <b>Supervisor–worker</b>  | One planner, many workers  | Separable subtasks   | Planner bottleneck |
|               | <b>Hierarchical</b>       | Supervisors of supervisors | Recursive splits     | Errors amplify     |
|               | <b>Pipeline / DAG</b>     | Each stage feeds the next  | Clear ordered stages | Errors propagate   |
|               | <b>Market / auction</b>   | Agents bid for tasks       | Dynamic allocation   | Bad bids           |
| Decentralized | <b>Blackboard</b>         | Shared workspace           | One shared artifact  | Stale state        |
|               | <b>Committee / debate</b> | Answer, then reconcile     | Judgment calls       | Groupthink         |
|               | <b>Swarm</b>              | Direct talk, no controller | Local negotiation    | $O(n^2)$ messages  |
| Hybrid        | <b>Dynamic topology</b>   | Rewires as work proceeds   | Unknown structure    | Hard to debug      |

# Task decomposition



# Task decomposition

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| <b>Data</b>       | Each agent handles a subset of documents or records |
| <b>Expertise</b>  | Security, performance, correctness, legal analysis  |
| <b>Stage</b>      | Plan, implement, test and review                    |
| <b>Hypothesis</b> | Agents investigate competing explanations           |
| <b>Tool</b>       | Web, database, code execution or internal documents |
| <b>Authority</b>  | Agents act in different administrative domains      |

**Example:** a 200-file PR split by data (files), then by expertise (security, perf).

# Things to consider when creating sub-tasks

Objective

Inputs and relevant context

Boundaries and exclusions

Available tools and permissions

Expected output schema

Completion test

Budget and deadline

Dependencies

## **Poor sub-task**

“Research storage systems”

## **Better sub-task**

“Identify storage-related papers published in 2026; return title, date, system contribution, evaluation scale and primary-source URL; exclude database query optimization”

# Granularity tradeoff

## Tasks too large

Little parallelism; the work stays sequential

## Tasks too small

Scheduling and communication overhead dominates

$$T_{\text{multi}} \approx T_{\text{planning}} + T_{\text{critical path}} + T_{\text{communication}} + T_{\text{merge}}$$

Parallelization helps only when the saved sequential work exceeds the planning, communication and integration costs.

**Example:** 20 one-line lookups batched into 4 subtasks to keep overhead below the gain.

# Static versus dynamic decomposition

## Static

Known before execution; predictable and easy to evaluate

*Example: summarize each of five uploaded PDFs*

## Dynamic

Agents create new tasks in response to discoveries

*Example: a dead link spawns a lookup task*

## Recursive

Workers may themselves delegate

*Example: a review agent spins up one sub-agent per subfield*

## Opportunistic

Idle agents claim newly available tasks

*Example: an idle agent picks up the next unlabeled document*

**Decomposition needs spawning limits, budgets, depth limits and termination conditions**

Without them, delegation explodes recursively.

# Common decomposition failures

## 01 Overlap



Two agents summarize the same paper

## 02 Gap



“2020–2023” plus “2025–2026” skips 2024

## 03 Hidden dependency



Pricing needs sizing, found only at run time

## 04 Unmergeable outputs

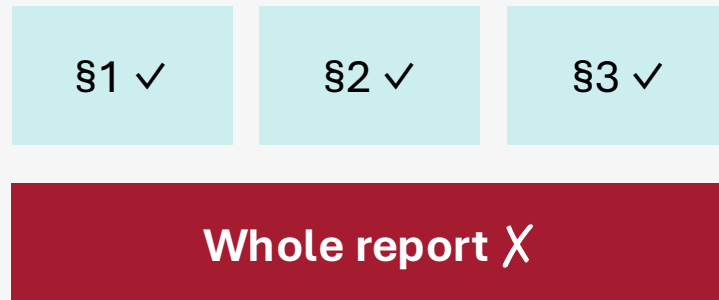


One agent returns free-form text, another returns JSON



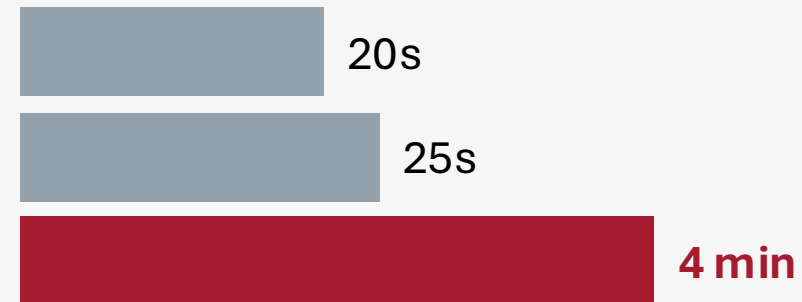
# Common decomposition failures, continued

## 05 Lost global context



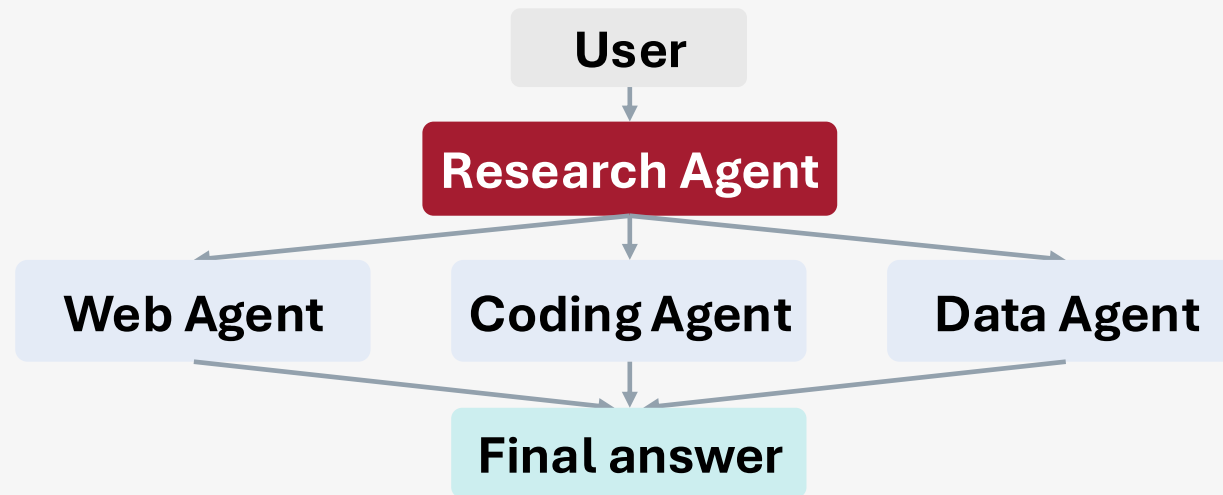
Every section is fine; the report contradicts itself

## 06 Straggler



Six agents finish in 20s, one takes 4 minutes

# Agent communication



# Four dimensions of multi-agent communication



## Topology

### Who can talk to whom?

supervisor–worker, peer-to-peer  
determined by architecture (discussed earlier)



## Medium

### How is information exchanged?

messages, shared memory, files, database



## Channel

### How is information passed?

p2p, broadcast, pub–sub, shared state, request–resp



## Format and Semantics

### How is a message structured and how to express information?

delegate, report, ask, handoff, cancel

# Communication medium



## Message passing

Agents send explicit messages over a queue or bus  
*e.g. coder posts a result to the orchestrator's inbox*



## Shared memory

One in-process state object every agent reads and writes  
*e.g. a shared task list updated as work completes*



## Files and artifacts (including DB)

Work products handed over through a file store  
*e.g. researcher writes notes.md; the writer opens it*

# Communication channels



## Direct point-to-point messages

Planner hands a task spec to the coder agent



## Broadcast

Orchestrator tells every agent the build failed



## Publish-subscribe

Agents subscribe to a “new ticket” topic and react



## Shared blackboard

Agents read and write one shared findings scratchpad



## Request-response

Agent queries a retrieval agent and waits for documents



## Artifact passing

One agent writes report.md; the next reads it

# Message format

## Natural language

Flexible, but ambiguous

## Structured messages

Easy to validate, less expressive

**A practical design uses both:**

```
{
  "task_id": "T17",
  "type": "result",
  "status": "complete",
  "claim": "The regression began after commit X",
  "confidence": 0.82,
  "evidence": ["log:2391", "commit:X"],
  "artifact": "analysis/T17.json",
  "requires_response": false
}
```

# Messages should carry semantics

## Separate control from content

### Everything in one string

```
{"message": "some text"}
```

```
{
  "id": "msg-381",
  "task_id": "t-17",
  "from": "planner",
  "to": "researcher",
  "type": "DELEGATE",
  "payload": { ... }
}
```

## Operations the runtime knows

DELEGATE

QUERY

REPORT

HANDOFF

UPDATE

CANCEL

ACK

ERROR

COMPLETE

The runtime can route, retry and stop without asking an LLM what the message meant.

Natural language for content, structured protocol for coordination.

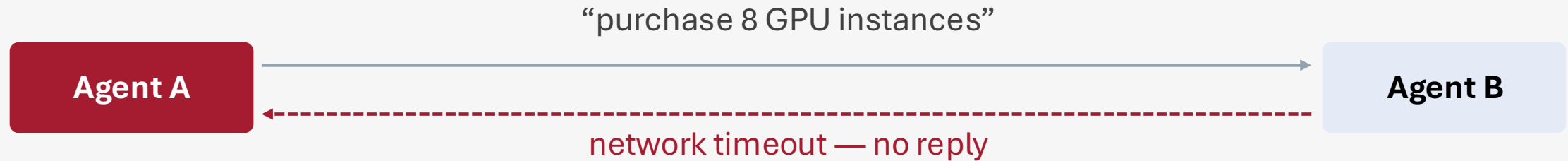
# Communication: the design space

| Dimension       | Options                                | Rule of thumb                      |
|-----------------|----------------------------------------|------------------------------------|
| <b>Topology</b> | supervisor–worker, peer-to-peer, mesh  | Mirror the task's dependencies     |
| <b>Medium</b>   | messages, shared memory, files         | One medium per link                |
| <b>Channel</b>  | p2p, broadcast, pub–sub, request–reply | Broadcast only when all must know  |
| <b>Format</b>   | natural language + control fields      | Route on the fields, not the prose |

Every choice trades latency, context or reliability.



# Communication is a hard distributed systems problem



## Did B execute the action?

- retry, or give up?
- at-most-once or at-least-once?
- is exactly-once even available here?
- idempotency key on the task\_id
- compensation / rollback path
- and who pays for a double purchase?

# Distributed systems are hard

## Unreliable network

Packet delay, drop, reordered, duplicated...

## No global clock

Clocks on different machines disagree

## Concurrency and non-determinism

Many machines modify state simultaneously

## Partial failure

You cannot tell a slow agent from a dead one

## Consistency among replicated states

Tradeoff between consistency, availability, partition, and latency

## Debugging and observability

Many requests, distributed logs with clock drift

Multi-agent systems inherit the common distributed systems problems.

# More challenges when the system runs multiple agents

## Scheduling

Who runs next, and for how long

## Resource allocation

Models, CPU, and other shared resources

## State management

Checkpoints, memory, provenance, and versioning

## No predetermined request flow

The system can't know *who* needs to run, for *how long*, or *how many resources* each agent will consume.

# Even harder when you consider the black-box LLMs!

## Context

### **Information loss**

Summaries drop  
what the worker  
knew

### **Context pollution**

What matters gets  
buried

### **Stale information**

B still acts on  $X=3$   
after  $X=5$

### **Conflicting beliefs**

A says safe, B says  
vulnerable

## Coordination and communication

### **Observability and failure diagnosis**

Why did it fail?

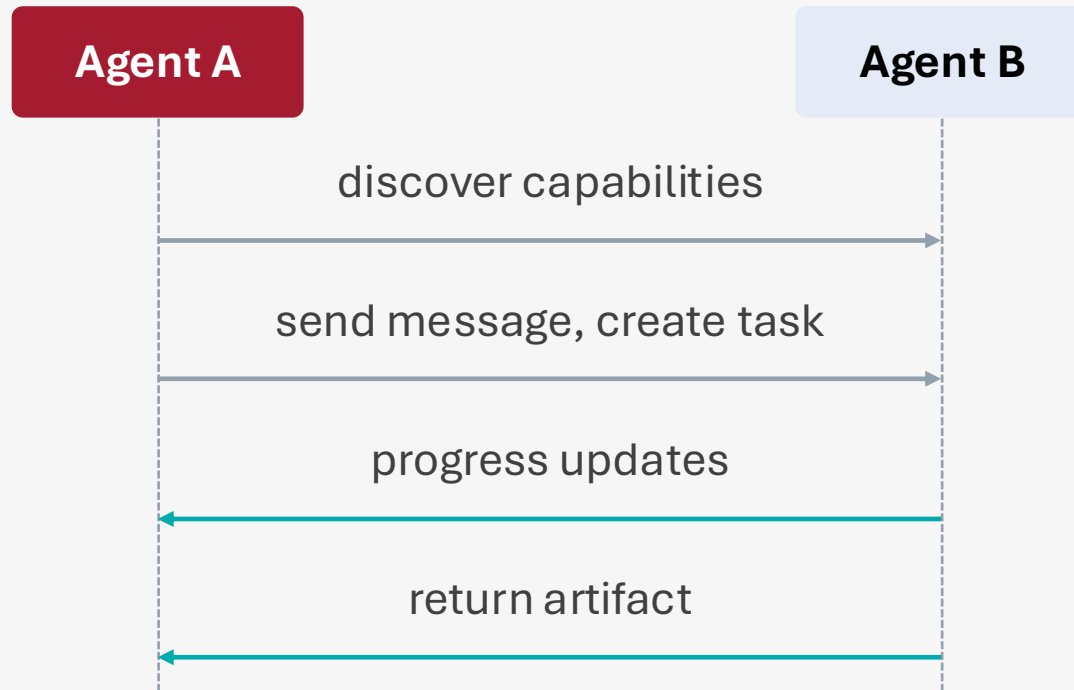
### **Lost ownership**

Each agent thinks another  
owns it

### **Communication loops**

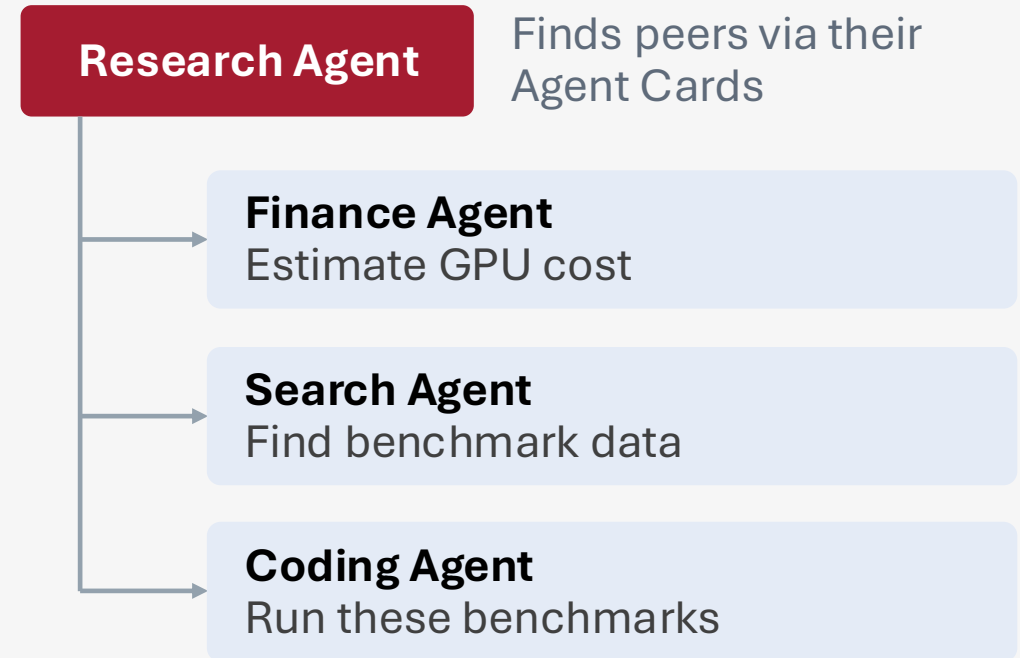
A asks B, B asks C, C asks A

# A2A: agent ↔ agent



## Example

“Analyze whether company X should move its inference stack to GPUs Y.”



# A2A agent card

Each A2A server publishes a machine-readable card: discovery plus a capability description.

```
{  
  "name": "Travel Agent",  
  "description": "Plans business travel",  
  "skills": [{ "id": "book-flight" }],  
  "supportedInterfaces": [ ... ],  
  "capabilities": { "streaming": true },  
  "securitySchemes": { ... }  
}
```

GET /.well-known/agent-card.json

## What a caller learns

**Identity** — who the agent is

**Skills** — what it claims it can do

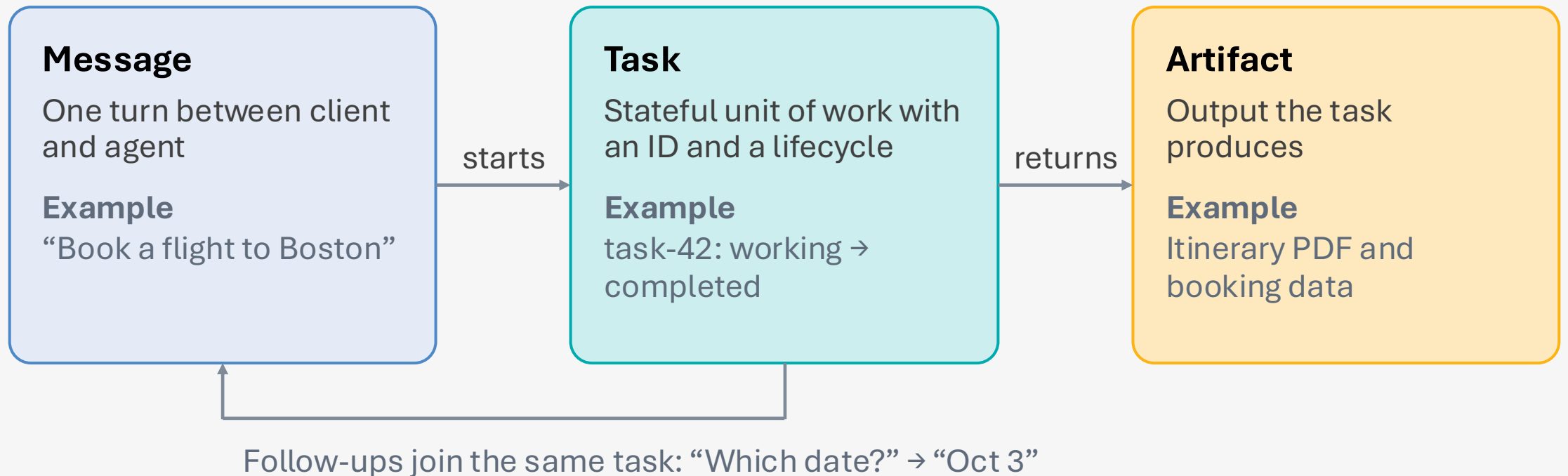
**Endpoints** — where and how to call

**Capabilities** — streaming, push

**Auth** — which security scheme to satisfy

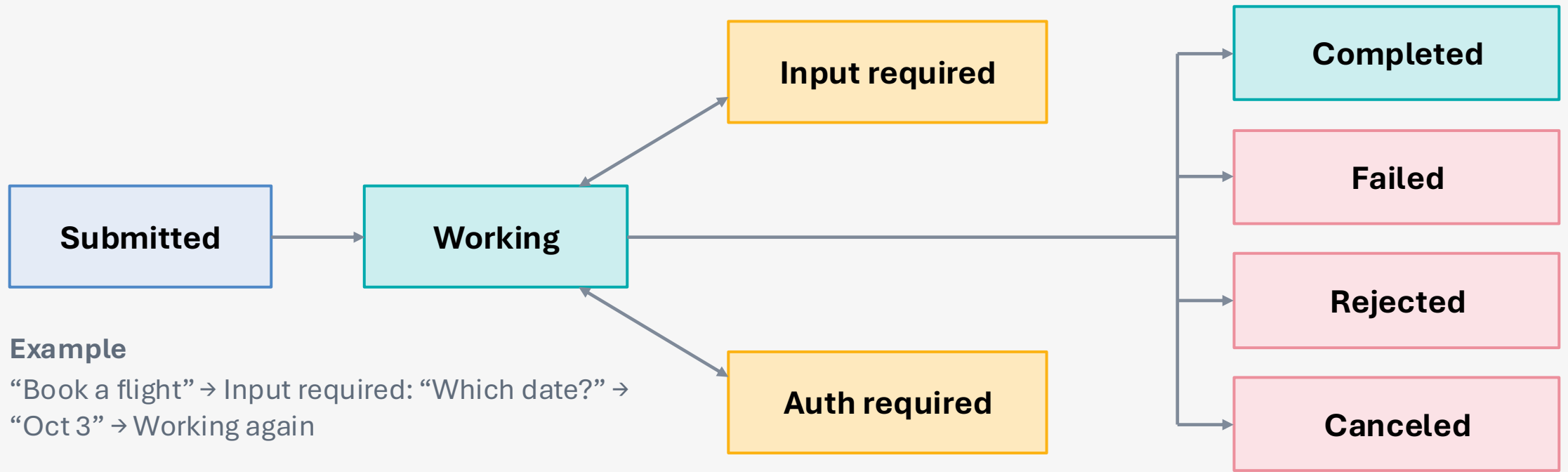
# Three A2A abstractions

A message starts a task; the task returns its results as artifacts.



# A2A task lifecycle

A2A is a stateful task protocol: work can pause for input or auth, then resume.





# Following a long-running task

Tasks can run for seconds or hours; the caller picks how to track them.

## Polling

Client re-checks until done

**Use for:** Short jobs

## Streaming

Streams live updates

**Use for:** Interactive output

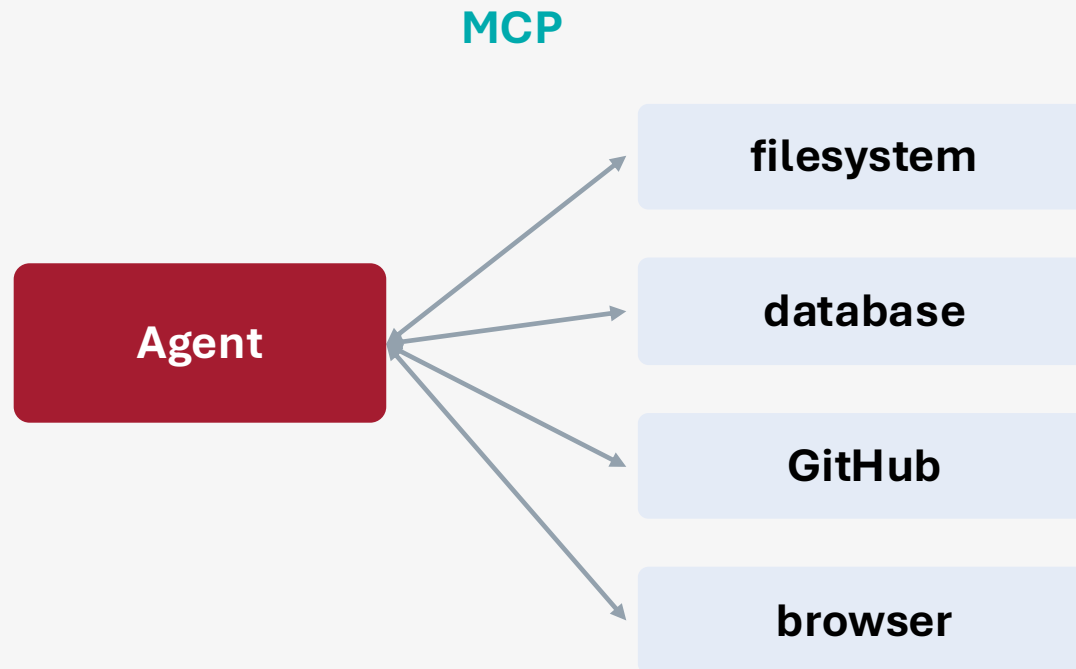
## Push notification

Webhook fires when done

**Use for:** Hours-long jobs

A2A standardizes communication, not orchestration: delegation, splitting and stopping stay with the harness.

# MCP: agent ↔ capability



## What MCP standardizes

- tools — callable, typed capabilities
- resources — readable data the host may attach
- prompts — reusable interaction templates
- one client speaks to many servers
- the host, not the model, decides what is exposed

MCP is the agent ↔ service boundary: one tool call, one result.

# MCP vs. A2A

|                     | MCP                       | A2A                    |
|---------------------|---------------------------|------------------------|
| Primary abstraction | capability / tool         | agent / task           |
| Relationship        | agent → service           | agent → agent          |
| Discovery           | tools, resources, prompts | agent capabilities     |
| Unit of work        | one tool invocation       | long-running task      |
| Internal state      | capability interface      | ideally opaque         |
| Analogy             | device / API interface    | service-to-service RPC |

One A2A remote agent may itself call twenty MCP servers.

# Design principles

## 1 Is multi-agent really necessary?

Single agent is sufficient, multiple agents often only adds overhead and complexity

## 2 Decide topology

reflect task dependencies, minimize communication

## 3 Identify information-loss points

treat aggregation nodes as places where detail drops

## 4 Identify failure-containment boundaries

Make sure the system does not stall

A dense mesh is rarely the best default. More communication can increase context pressure without increasing useful information.

# Next class

- Agent design III
  - Agent memory
  - Agent reliability
  - Agent evaluation

